Fundamental Programming in JAVA

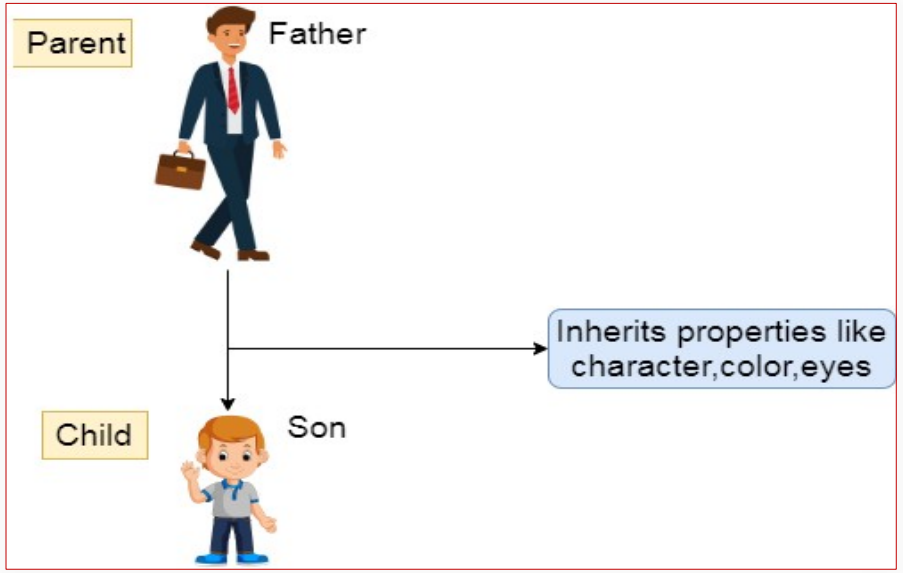
INHERITANCE

Objectives

- To define a subclass from a superclass through inheritance.
- To invoke the superclass's constructors and methods using the **super** keyword .
- To override instance methods in the subclass.
- To distinguish differences between overriding and overloading.
- To explore the **toString()** method in the **Object** class.
- To explore the **equals** method in the **Object** class.
- To enable data and methods in a superclass accessible from subclasses using the **protected** visibility modifier.
- To prevent class extending and method overriding using the **final** modifier.

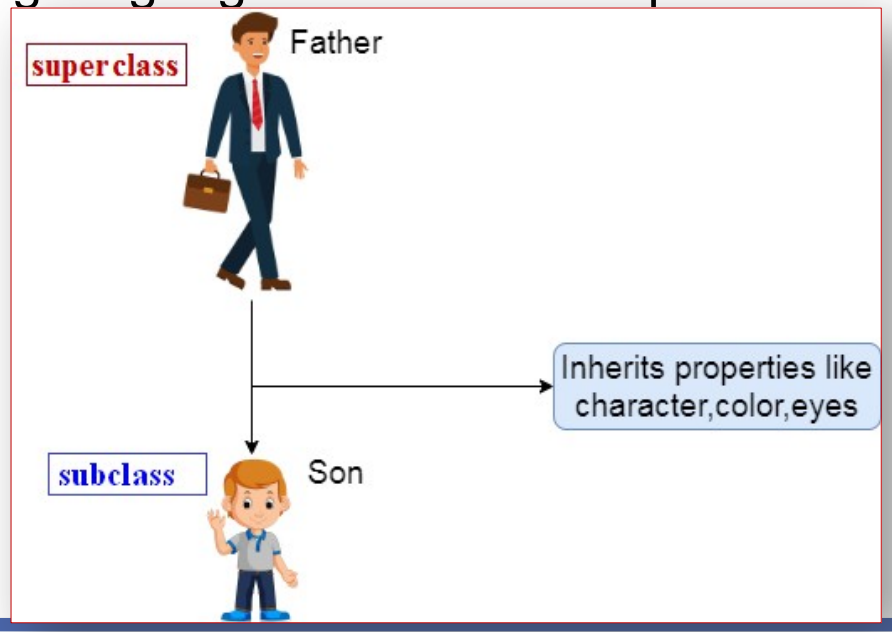
What is inheritance ?

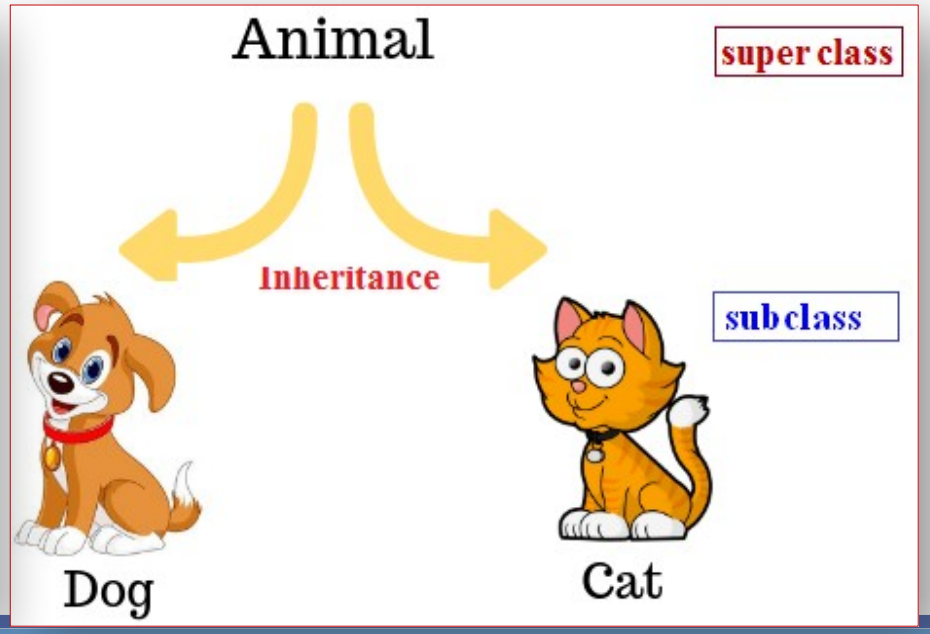
- Inheritance in Java is **a concept that acquires the properties from one class to other classes**. With inheritance, programmers save time during program development by reusing proven and debugged high-quality software. This also increases the likelihood that a system will be implemented effectively.



What is inheritance ?

- When creating a class, rather than declaring completely new members, you can designate that the new class should inherit the members of an existing class.
  - The existing class is called the **superclass**, and the new class is the **subclass**.
  - (The C++ programming language refers to the superclass as the **base class** and the subclass as the **derived class**).





What is inheritance ?

- Each **subclass** can become a **superclass** for future **subclasses**.
- **Subclasses** can add their own fields and methods. Therefore, **subclasses** are more specific than their **superclass** and represents a more specialized group of objects.
- NOTE: in Java **every class has one and only one direct superclass.** *(explained more later)*

What is inheritance ?

- Typically, the **subclass** exhibits the behaviors of its **superclass** and additional behaviors that are specific to the **subclass**.

```
public class Vehicle {
    ..............
    ..............
}
                IS-A Relationship

public class Car extends Vehicle {
    ..............
    ..............
}
                IS-A Relationship

public class Alto extends Car {
    ..............
    ..............
}
```

Fig a: Is-A relationship between classes

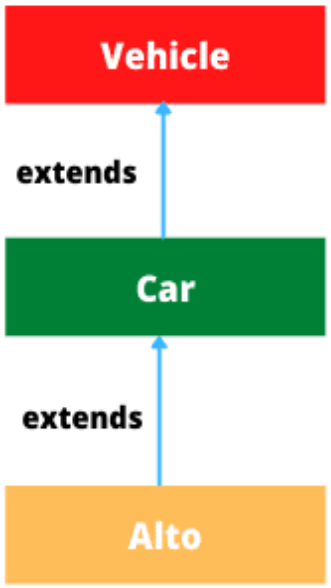


Fig b: Inheritance hierarchy for Vehicle, Car, and Alto

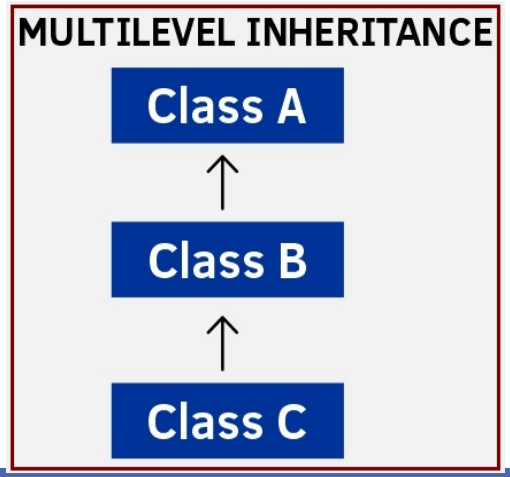
Direct inheritance and indirect inheritance

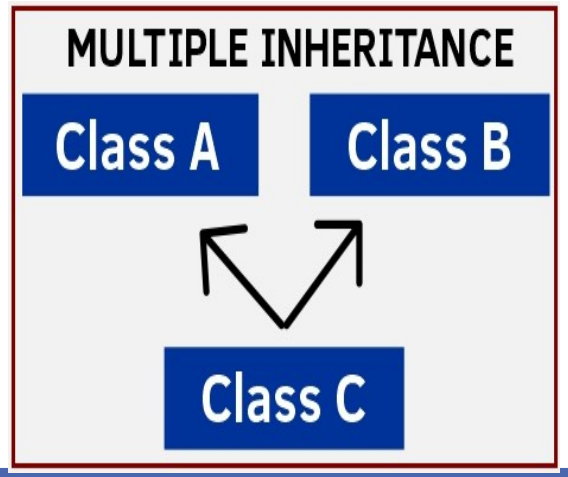
- A direct **superclass** is the super class from which the **subclass** explicitly inherits.
- An indirect **superclass** is any class above the direct **superclass** in the class hierarchy, which defines the inheritance relationships between classes.
  - In fact, every class you define in Java is inherited from an existing class, either **explicitly** or **implicitly**.
  - All the classes that we created were all extended **implicitly** from (**java.lang.Object**).

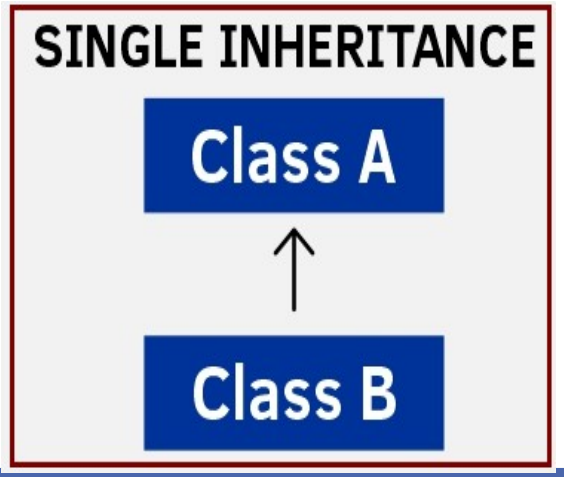
Single and multiple inheritance

- When a subclass is derived from one direct superclass it is called single inheritance.
- When a subclass is derived from more than one direct superclass it is called
- Multiple inheritance.

Note: Java, unlike C++, does not support multiple inheritance.







Inheritance Example

- Imagine the positions Programmer and Manager within an organization.
- **Both positions have a common set of properties**, including **name**, **address**, and **phone number**.
- These positions **also have different properties.**
- A Programmer may be concerned with a project's programming languages, whereas a Manager may be concerned with project status reports.

Programmer

name
address
phoneNumber
experience
programmingLanguages
writeCode()

```
class Programmer {
    String name;
    String address;
    String phoneNumber;
    float experience;
    String[] programmingLanguages;
    void writeCode() {}
}
```

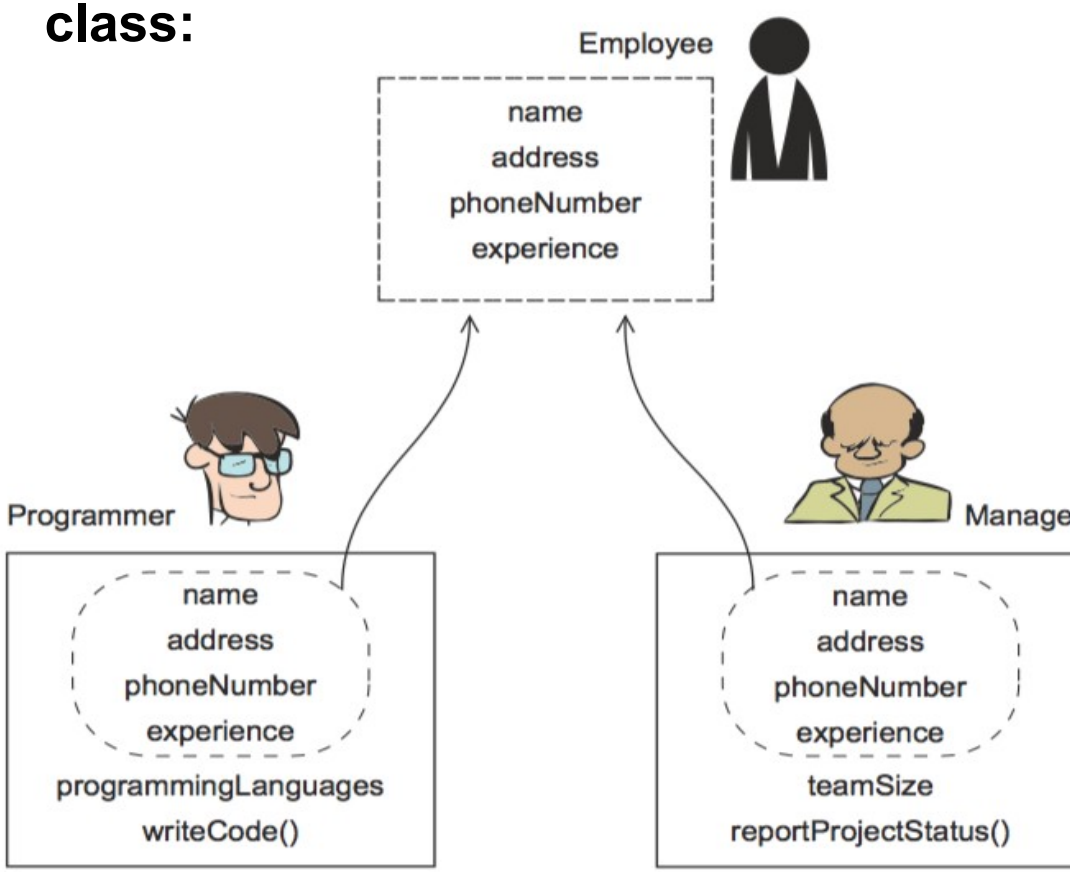
Manager

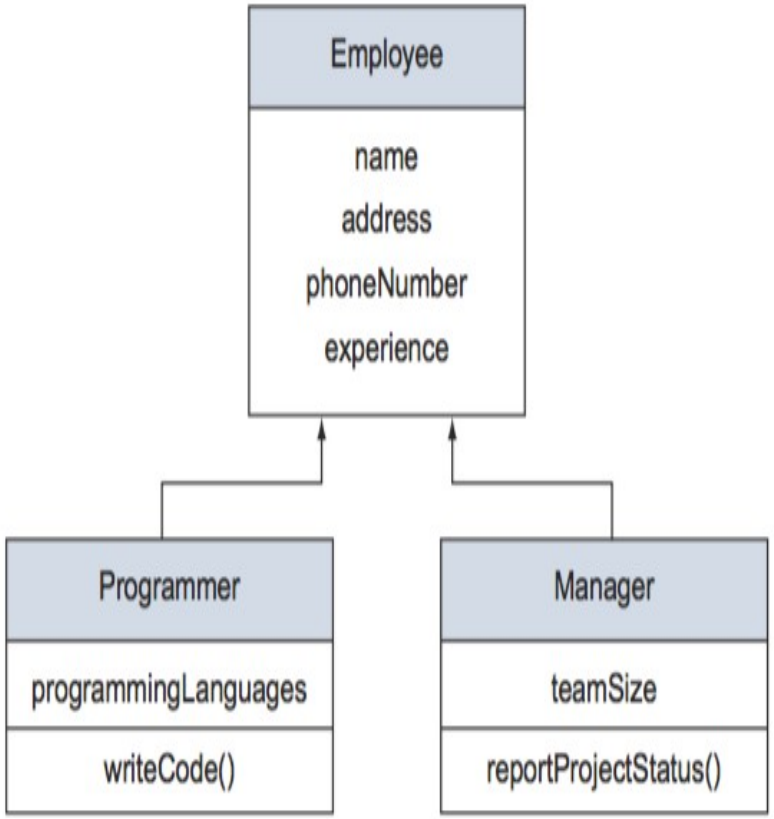
name
address
phoneNumber
experience
teamSize
reportProjectStatus()

```
class Manager {
    String name;
    String address;
    String phoneNumber;
    float experience;
    int teamSize;
    void reportProjectStatus() {}
}
```

Inheritance Example

Put the common information in a base(general) class:



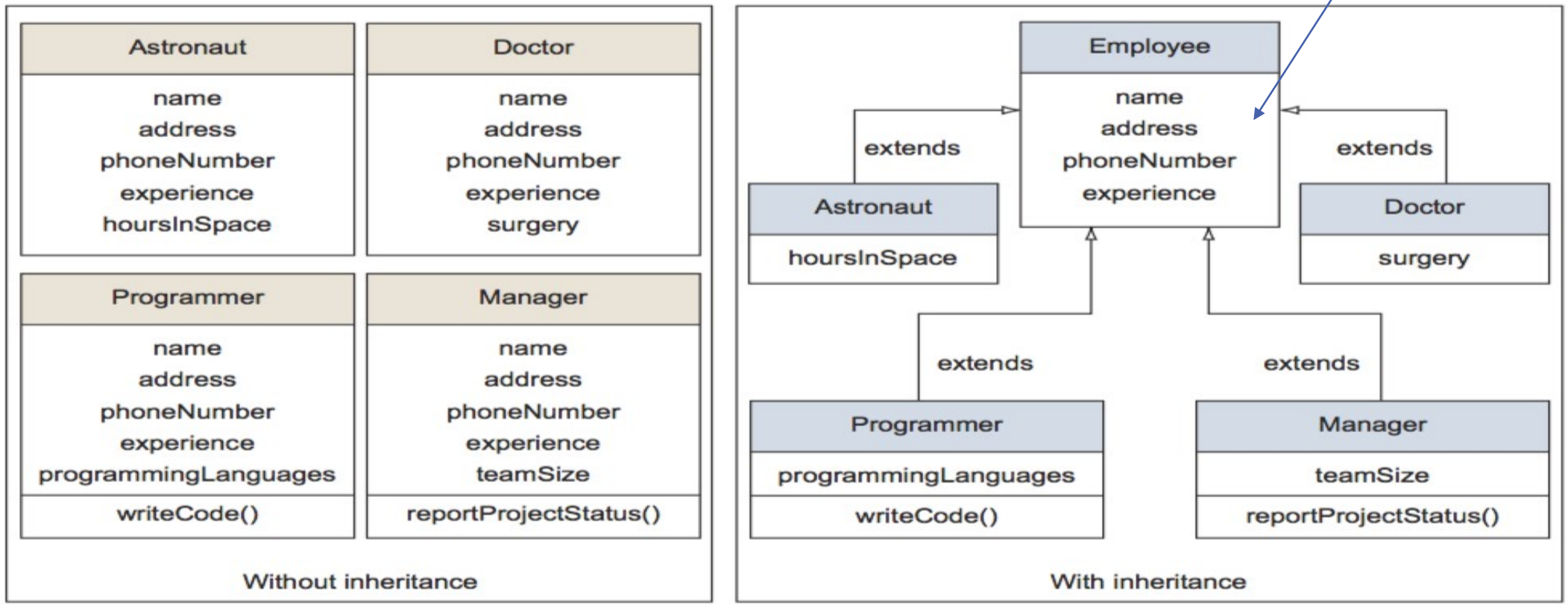



```
class Employee {
    String name;
    String address;
    String phoneNumber;
    float experience;
}
class Programmer extends Employee {
    String[] programmingLanguages;
    void writeCode() {}
}
class Manager extends Employee {
    int teamSize;
    void reportProjectStatus() {}
}
```

Inheritance Benefits

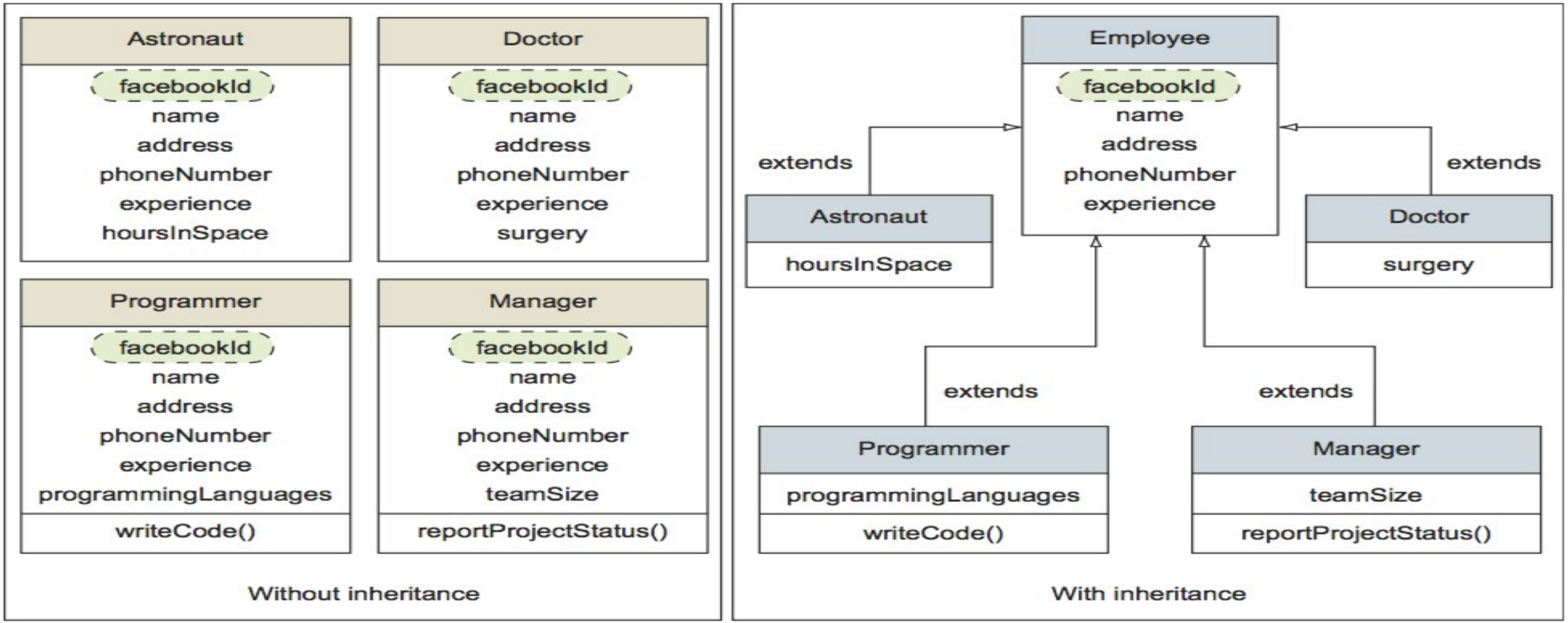
- **Less code redundancy:**

Common list of attributes among all subclasses



Inheritance Benefits

- **Ease of Modification**: Suppose you need to add new attribute **(facebookid)**.



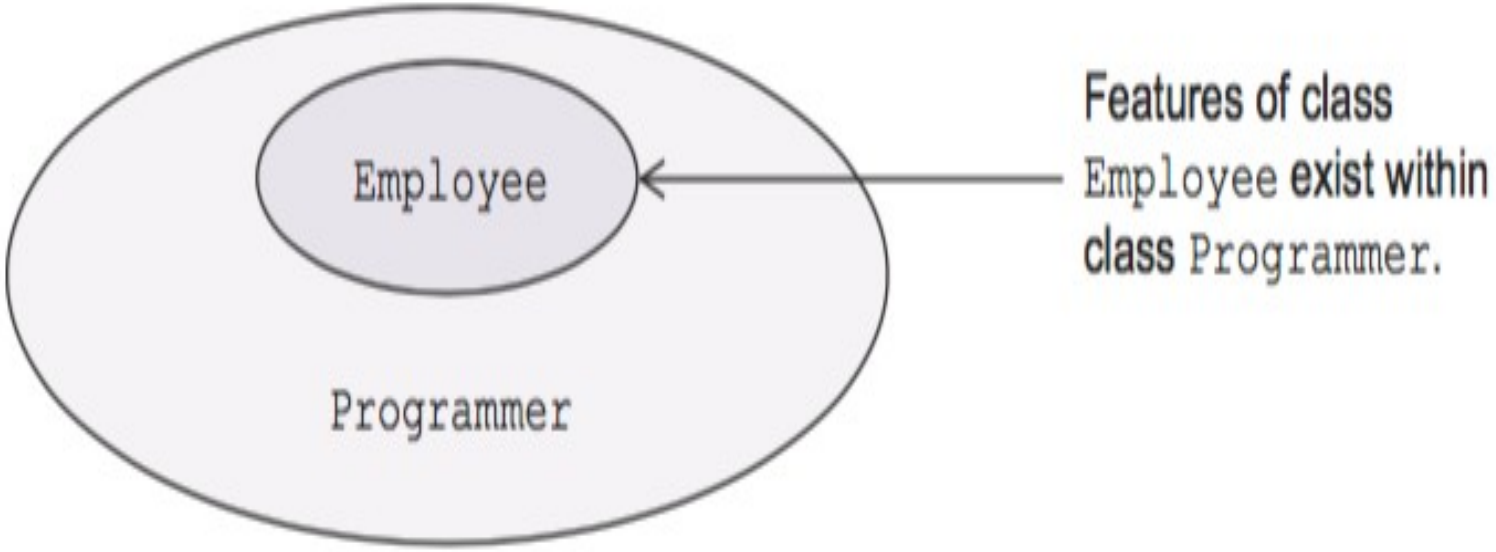
Inheritance Benefits

- **Extensibility**: Code that works with the base class in a hierarchy tree can work with all classes that are added using inheritance later.
  - Assume that an organization needs to **send out invitations** to all its employees and that it uses the following method to do so:

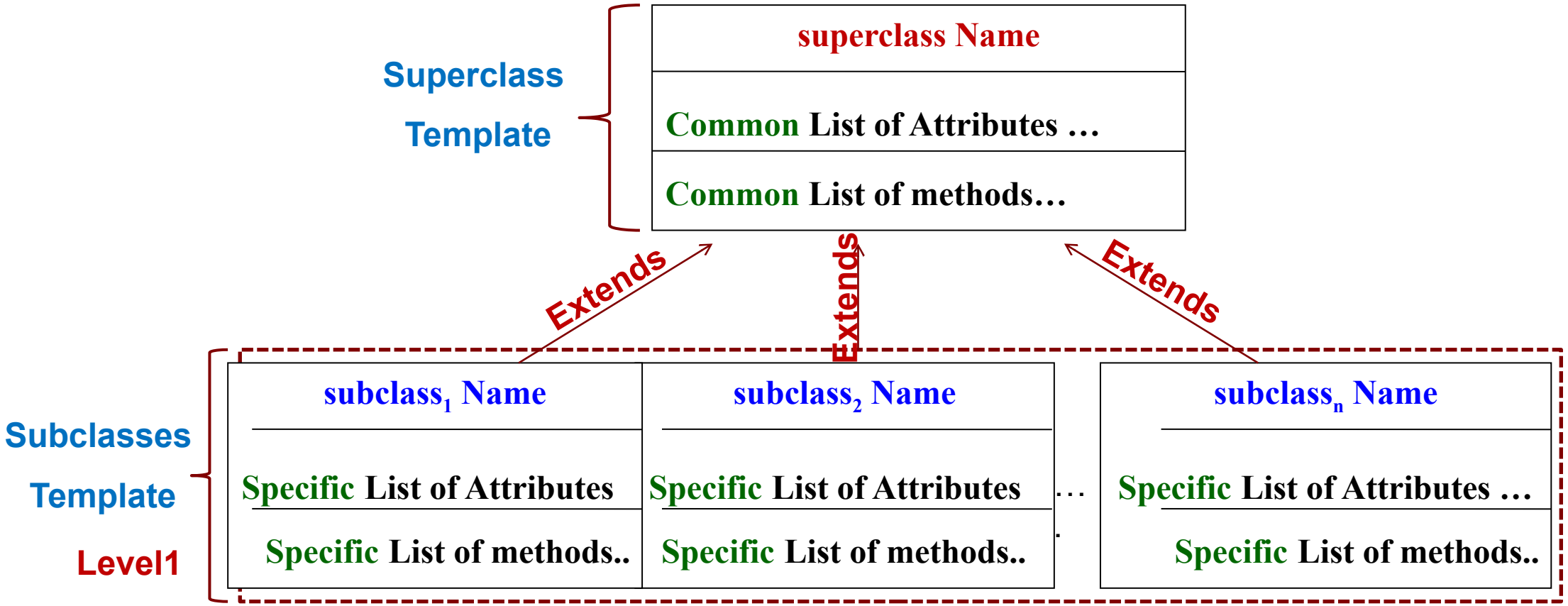
```
class HR {
    void sendInvitation(Employee emp) {
        System.out.println("Send invitation to" +
                                emp.name + " at " + emp.address);
    }
}
```

Inheritance Benefits

- Because the method **sendInvitation** accepts an argument of type Employee, you can also pass to it a subclass of Employee. Essentially, this design means that **you can use the previous method with a class defined later that has Employee as its base class.** Inheritance makes code extensible.



Inheritance (Design Phase)



Using The Keyword extends

- The keyword **extends** is used for inheritance.
- Examples:

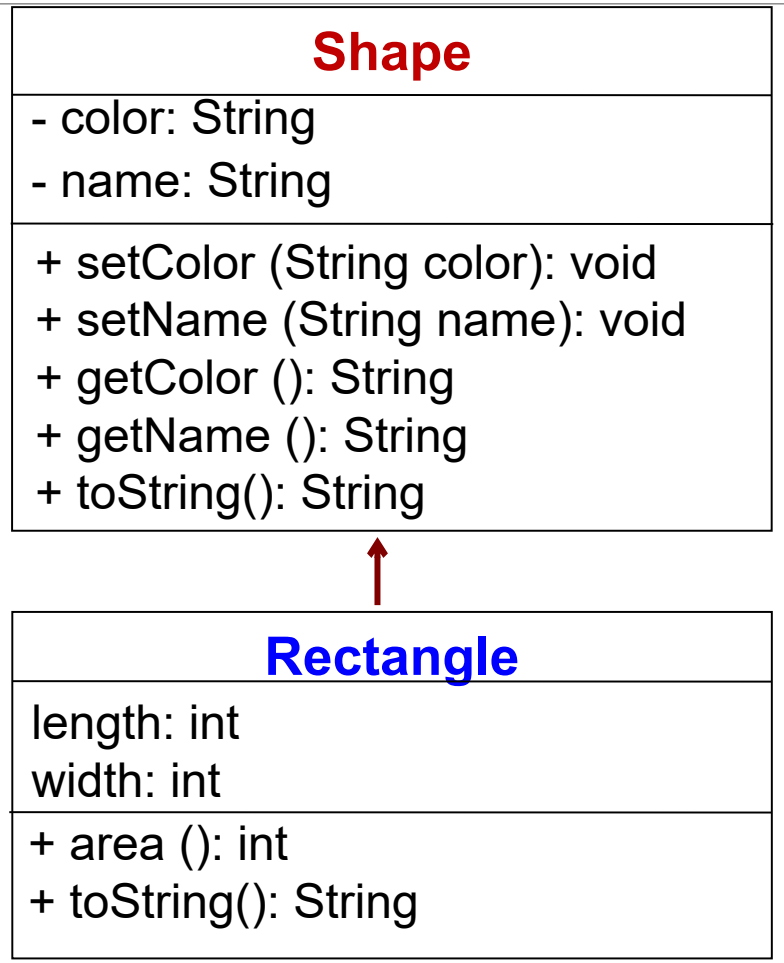
```
class Car extends Vehicle {….. }
```

//Car is subclass, while Vehicle is superclass

```
class Circle extends Shape {…. }
```

//Circle is subclass, while Shape is superclass

Inheritance Example (Design Phase)



Inheritance Example (Implementation Phase)

```java
public class Shape {
   private String color,name;

    public void setColor(String color) {
        this.color = color;
    }

    public String getColor() {return color; }

    public String getName() {return name;}

    public void setName(String name) {this.name = name;}
   @Override
    public String toString() {return "Shape{" + "color=" + color +
            ", name=" + name + '}';}

}
```

```java
public class Rectangle extends Shape {
    int length, width;
    public Rectangle() {
        length = 0;
        width = 0;
        System.out.println(" length=" + length + "  width= " + width);}

    public Rectangle(int length, int width) {
        this.length = length;
        this.width = width; }

    public int area() {
        return length * width;}

    @Override
    public String toString() {
        return "Rectangle{" + "length=" + length + ", width=" + width + '}'; }
}
```

Inheritance Example (*explanation)*

- In the previous inheritance example code:
- The Rectangle subclass can access all the public and protected members in the Shape class (superclass). On the contrary, the Shape superclass can access nothing from the Rectangle subclass.
- The Shape superclass has two data fields (color and name) and five methods (setColor(), setName(), getColor (), getName(), and toString()).
- The Rectangle subclass has four data fields (color, name, length, and width) and seven methods (all the superclass methods, area() and toString()).
- The extends keyword indicates that a class is inherited from another class.

Inheritance Explanation

- The following steps are taken by the Java compiler when creating an object from a superclass, a subclass, or both of them:
  - The process of creating an object from the **superclass** implements **only** the **superclass** constructor without invoking the subclass(es).
  - Creating an object from a subclass **implicitly** calls the no-argument constructor in the superclass first and then implements the used constructor in the subclass.
  - If the superclass doesn't have a no-argument constructor and only has constructors with arguments, then the subclass constructor must **explicitly** call one of the superclass constructors.

Inheritance Example of (main class)

```
public class Main {
    public static void main(String[] args) {
        Shape s1 = new Shape();
        Rectangle r1 = new Rectangle();
        System.out.println("s= " + s1.toString() +
                "\nr1=" + r1.toString());
    }
}
```

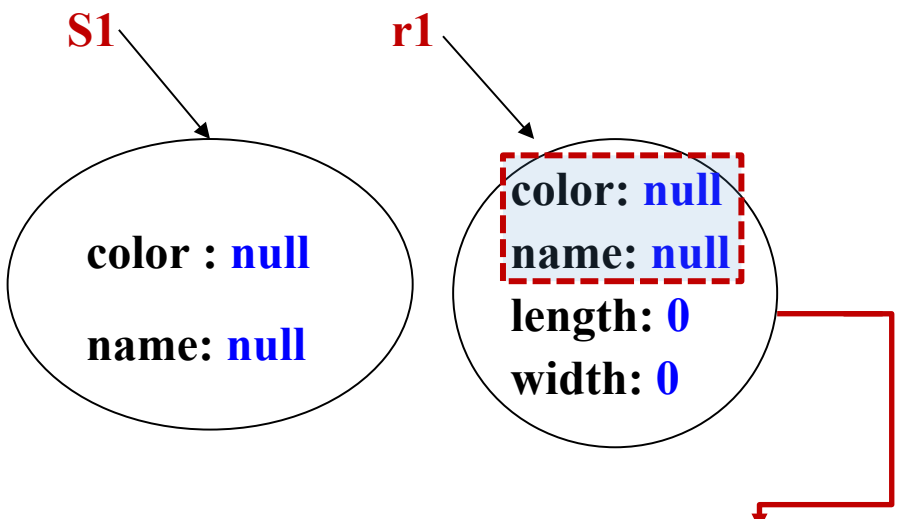
- Creating an object from the superclass (Shape) in line 4 should **only** implements the required constructor (Note that there is no zero constructor in the Shape class, so the default constructor will be invoked).
- Creating an object from the subclass (Rectangle) in line 5 should implement the constructor without parameters in the Rectangle class in line3. But before that, the default constructor in the (Shape) class should be invoked to provide the default values for the data fields of the superclass, and then complete implementing the constructor in line 3.

Inheritance Example of (main class)

```java
public class Main {
    public static void main(String[] args) {
        Shape s1 = new Shape();
        Rectangle r1 = new Rectangle();
        System.out.println("s= " + s1.toString() +
                "\nr1=" + r1.toString());
    }
}
```

Output:

```
length=0  width= 0
s= Shape{color=null, name=null}
r1=Rectangle{length=0, width=0}
BUILD SUCCESSFUL (total time: 1 second)
```



These values are filled by contacting the superclass's default constructor (since there are no constructors in the superclass) as a first step before implementing the required constructor (Rectangle()) in the subclass (Rectangle class).

Inheritance (Constructors)

But what if we add the following constructor to the class (Shape) ???

Followed up in the next slide

```java
1   public class Shape {
2
3       private String color, name;
4
5       public Shape(String color) {
6           this.color = color;
7       }
8
9       public void setColor(String color) {
10          this.color = color;    }
11
12      public String getColor() {
13          return color;    }
14
15      public String getName() {
16          return name;   }
17
18      public void setName(String name) {
19          this.name = name;     }
20
        @Override
22      public String toString() {
23          return "Shape{" + "color=" + color +
24                  ", name=" + name + '}';     }
25  }
```

Inheritance (Constructors)

Lots of errors occurred in the subclass (Rectangle):

```java
public class Rectangle extends Shape {
    int length, width;
    public Rectangle() {
        length = 0;
        width = 0;
        System.out.println("length=" +
        length + "  width= " + width);}

    public Rectangle(int length, int width) {
        this.length = length;
        this.width = width; }

    public int area() {
        return length * width;}

    @Override
    public String toString() {
        return "Rectangle{" + "length=" + length
                + ", width=" + width + '}'; }
}
```

In this case error messages appear in **all the constructors** in the subclass saying (cannot find constructor Shape()). This is because the default constructor in the superclass has no longer existed and there is no constructor without parameters in the superclass. **To solve this problem,** We have **two ways**, and we will explain each of them in detail….

Using the keyword super

- The keyword super refers to the superclass of the class in which super appears. This keyword can be used in three ways:
  - To call a superclass instance variables.
  - To call a superclass methods.
  - To call a superclass constructors.
- The syntax of the (super) to call a superclass construct is:
  - super(), or super(parameter list) just like this( ) keyword, super should be written as the first statement in the constructor.

Using the keyword super to invoke superclass constructor

```java
public class Shape {
    private String color, name;

    public Shape() {   }

    public Shape(String color) {
        this.color = color;     }

    public Shape(String color, String name) {
        this.color = color;
        this.name = name;     }

    public void setColor(String color) {
        this.color = color;     }

    public String getColor() {
        return color;     }

    public String getName() {
        return name;     }

    public void setName(String name) {
        this.name = name;     }
    @Override
    public String toString() {
        return "Shape{" + "color=" + color +
                ", name=" + name + '}';     }
}
```

```java
public class Rectangle extends Shape {
    int length, width;
    public Rectangle() {
        super("Black");
        length = 0;
        width = 0;
        System.out.println("length=" +
                length + "  width= " + width);  }

    public Rectangle(String color, String name, int length, int width) {
        super(color, name);
        this.length = length;
        this.width = width;   }

    public int area() {
        return length * width;  }

    @Override
    public String toString() {
        return "Rectangle{" + "length=" + length + ", width=" + width
                +", Color="+getColor()+", Name="+getName()+ '}';
    }
}
```

Using the keyword super

```
1   public class Main {
2
3       public static void main(String[] args) {
4           Shape s1 = new Shape();
5           Rectangle r1 = new Rectangle("red", "Rectangle",50,100);
6           System.out.println("   r1=" + r1.toString());
7       }}
```

As shown in **Main** class,

- The process of creating (r1) using the constructor with 4 parameters (line 10) requires calling the constructor with 2 parameters in the superclass (line 9), passing the actual values ("red", "Rectangle") to it.

Inheritance – Calling superclass methods

```java
public class Rectangle extends Shape {
    int length, width;
    public Rectangle() {
        super("Black");
        length = 0;
        width = 0;
        System.out.println("length=" +
                length + "  width= " + width);  }

    public Rectangle(String color, String name, int length, int width) {
        super(color, name);
        this.length = length;
        this.width = width;   }

    public int area() {
        return length * width;  }

    @Override
    public String toString() {
        return super.toString()+ "Rectangle{" + "length=" + length +
                ", width=" + width;}
}
```

To invoke a method in the superclass. The code of the (toString) method in the subclass could be written as follows:

The (**super.toString()**) implements the superclass's (toString) method and the returned value added to the implementation of the rest of the string.

Inheritance main class

```java
public class Main {

    public static void main(String[] args) {
        Shape s1 = new Shape();
        Rectangle r1 = new Rectangle("red", "Rectangle",50,100);
        System.out.println("   r1=" + r1.toString());
    }}
```

Output:

```
   r1=Shape{color=red, name=Rectangle}Rectangle{length=50, width=100

BUILD SUCCESSFUL (total time: 0 seconds)
```

Are superclass’s Constructor Inherited?

- No. They are not inherited,they are invoked explicitly or implicitly.
- Explicitly using the super keyword.
- A constructor is used to construct an instance of a class.
- Unlike properties and methods, a superclass's constructors are not inherited in the subclass.
- They can only be invoked from the subclasses' constructors, using the keyword super.
- *If the keyword super is not explicitly used, the superclass's no-arg constructor is automatically invoked.*

Constructor Chaining

```
class Point {
Point()
{          System.out.print("Poin
t Constructor ==> " );
}}
```

```
class Circle extends Point{
public Circle(){
System.out.print("Circle
Constructor ==> " ); } }
```

```
class Cylinder extends Circle {
public Cylinder()
{  System.out.print("Cylinder
Constructor." ); } }
```

```
class Project {
public static void main(String[]
args) {
Cylinder ob = new
Cylinder();     }}
```

```
Output:
Point Constructor ==> Circle Constructor ==> Cylinder Constructor.
```

Constructor Chaining (Code Tracing)

1

An object of type Cylinder is created using the empty constructor

```
class Project {
public static void main(String[] args)
{
Cylinder obj = new Cylinder(); }
}
```

Constructor Chaining (Code Tracing)

2

The output statement of the **Point** class in the empty constructer is executed

```
class Point {
Point(){
System.out.print("Point Constructor ==> " );
}}
```

```
Output:
Point Constructor ==>
```

Constructor Chaining (Code Tracing)

3

The output statement of the **Circle** class in the empty constructer is executed

```
class Circle extends Point{
public Circle(){
 System.out.print("Circle Constructor ==> " );
} }
```

```
Output:
Point Constructor ==> Circle Constructor ==>
```

Constructor Chaining (Code Tracing)

4

The output statement of the **Cylinder** class in the empty constructer is executed

```
class Cylinder extends Circle {
public Cylinder(){
 System.out.print("Cylinder Constructor ==>
" );
```

```
Output:
Point Constructor ==> Circle Constructor ==> Cylinder Constructor.
```

The protected Modifier

- The **protected** modifier can be applied on data and methods in a class. A protected data or a protected method in a public class can be accessed by any class in the same package or its subclasses, even if the subclasses are in a different package.

- Access modifiers: **private, default, protected and public.**

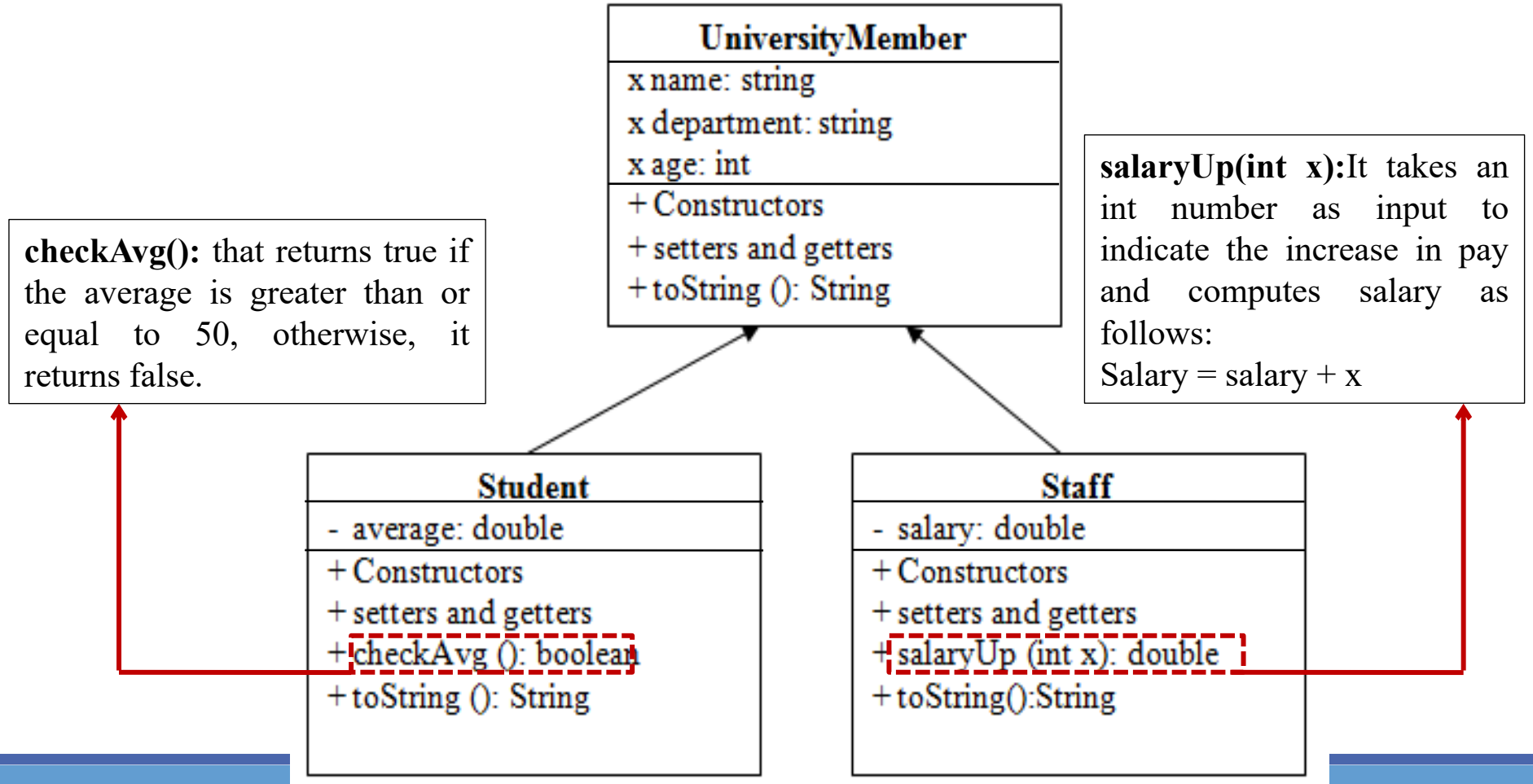
Access Modifiers Summary

Modifier members in class	Accessed from the same class	Accessed from the same package	Accessed from a subclass	Accessed from a different package
public	✓	✓	✓	✓
protected	✓	✓	✓	—
default	✓	✓	—	—
private	✓	—	—	—

Access Modifiers Summary

```
package p1;

public class C1 {
    public int x;
    protected int y;
    int z;
    private int u;

    protected void m() {
    }
}

public class C2 {
    C1 o = new C1();
    can access o.x;
    can access o.y;
    can access o.z;
    cannot access o.u;

    can invoke o.m();
}

package p2;

public class C3
        extends C1 {
    can access x;
    can access y;
    can access z;
    cannot access u;

    can invoke m();
}

public class C4
        extends C1 {
    can access x;
    can access y;
    cannot access z;
    cannot access u;

    can invoke m();
}

public class C5 {
    C1 o = new C1();
    can access o.x;
    cannot access o.y;
    cannot access o.z;
    cannot access o.u;

    cannot invoke o.m();
}
```

Exercise: Design the following classes and write the given methods:



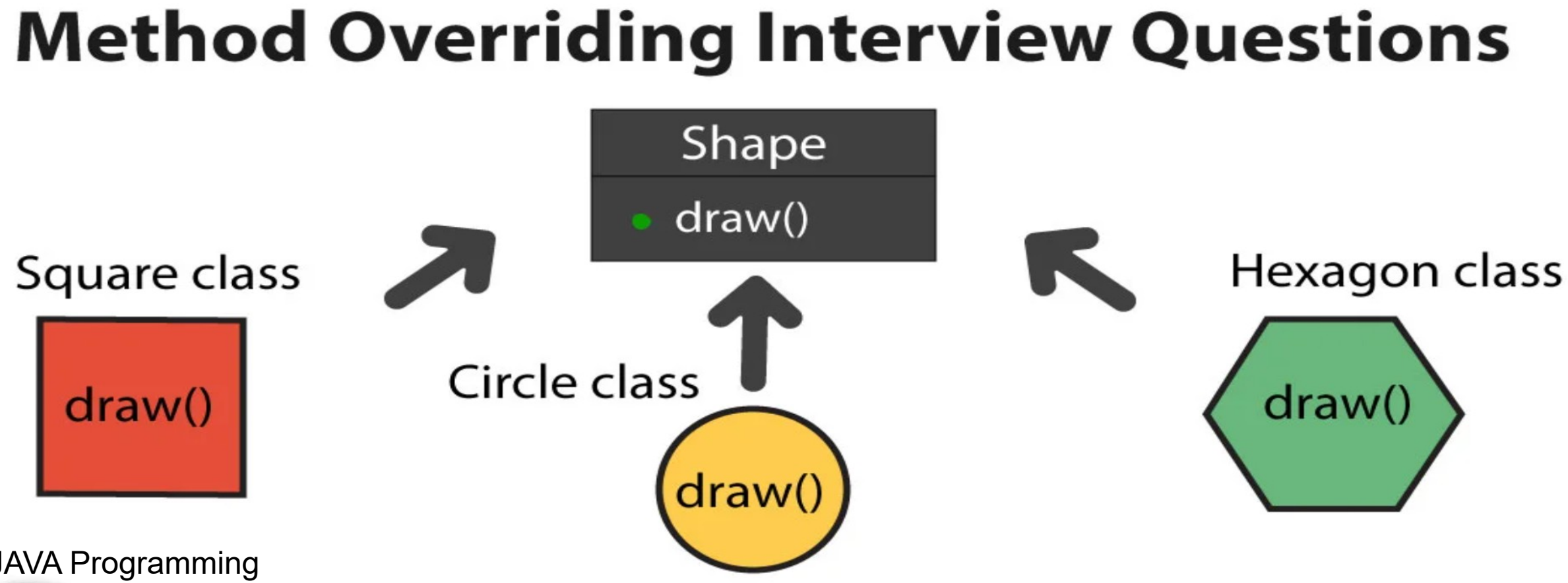
Examples of possible inheritance relationship between different classes

Superclass	Subclasses
Student	GraduateStudent, UndergraduateStudent
Shape	Circle, Triangle, Rectangle
Loan	CarLoan, HomeImprovementLoan, MortgageLoan
Employee	Faculty, Staff
BankAccount	CheckingAccount, SavingsAccount

Overriding & Overloading methods

- A subclass inherits methods from superclass. Sometimes it is necessary for the subclass to modify the implementation of a method defined in the superclass. This is referred to as method overriding.
  - The (toString) method is an example of overriding in the previous examples. The signature of the overridden methods should be exactly the same but in different classes.
  - On the other hand, overloading is the reuse of the method name in the same class but with different signature to distinguish between them. Constructors are an example of overloading.

Overriding & Overloading methods

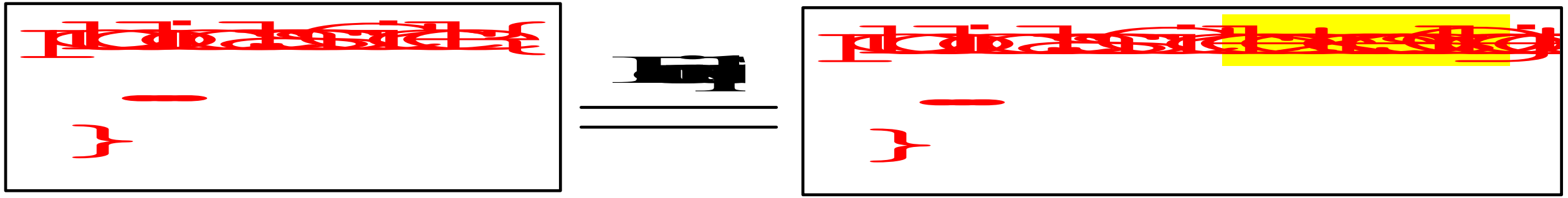


Overriding & Overloading methods

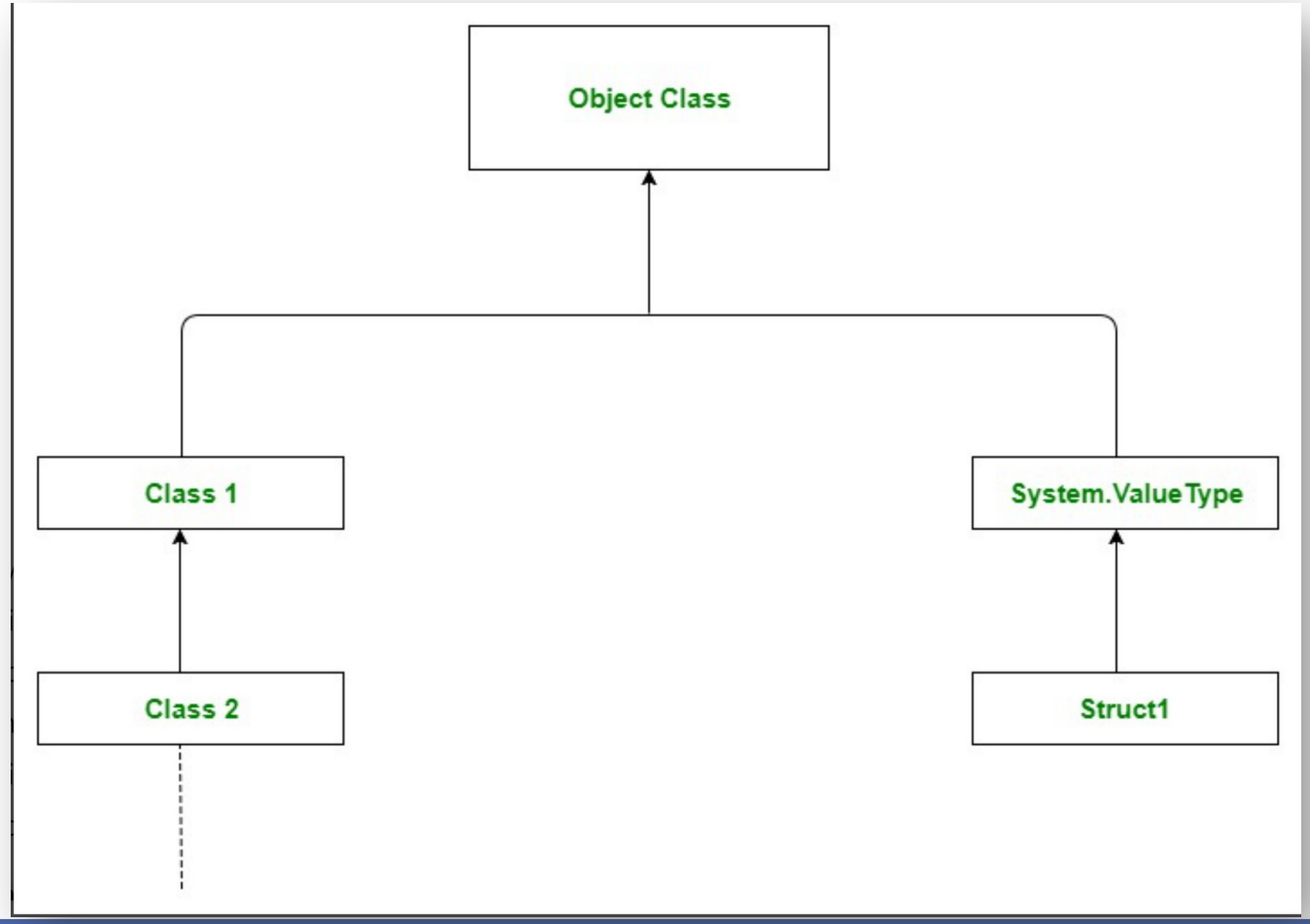
Overloading	Overriding
Must have at least two methods by the same name in the same class	Must have at least one method by the same name in both superclass and subclass
Must have a different number of parameters	Must have the same number of parameters
If the number of parameters is the same then must have different types of parameters	The type of parameters also must be the same
If the number and type of parameters are the same then must have different orders of parameters	The order of parameters also must be the same
Constructors are an example of overloading	The (**toString**) method is an example of overriding
Overloadingis known as a static polymorphism	Overriding is known as a dynamic polymorphism

The Object Class and Its Methods

- Every class in Java is descended from the **java.lang.Object class**. If no inheritance is specified when a class is defined, the superclass of the class is Object.
- ***Note:*** *Object class acts as a root of inheritance hierarchy in any java program.*



The Object Class and Its Methods



The toString() method in Object

- The toString() method returns a **string representation** of the object.
- Note: Default behavior of toString() is to print class name, then @, then unsigned hexadecimal representation of the hash code of the object.
- Example :

```
Loan loan = new Loan();
System.out.println(loan.toString());
```

- The code displays something like Loan@15037e5 . This message is not very helpful or informative. Usually **you should override the toString method** so that it returns a digestible string representation of the object.

The equals() Method

- The **`equals()`** method compares the contents of two objects. The default implementation of the equals method in the Object class is as follows:

```
public boolean equals(Object obj) {
  return this == obj;
}
```

- For example, the equals method is overridden in the Rectangle class.

```
public boolean equals(Rectangle obj) {
   return (this.width==obj.width) &&
          (this.length==obj.length);
   }
```

NOTE

- The == comparison operator is used for comparing two primitive data type values or for determining whether two objects have the same references.
- The equals method is intended to test whether two objects have the same contents, provided that the method is modified in the defining class of the objects.
- The == operator is stronger than the equals method, in that the == operator checks whether the two reference variables refer to the same object.

The final Modifier

- The final class cannot be extended:

```
final class Math {
        ...
    }
```

- The **`final`** variable is a constant:

```
final static double PI = 3.14159;
```

- The **`final`** method cannot be overridden by its subclasses.

The final Method

```
1   class Emp  {
2           protected String name;  protected String job;  protected int startYear;
3           public Emp(){ name="unknown"; job="unknown"; startYear=0;       }
4           public Emp(String name){ this(); this.name = name;}
5           public Emp(String name, String job) { this(name);this.job=job;}
6           public Emp(String name, String job, int sy){
7                this(name,job); startYear=sy;}
8
9           public final int calYearServices(int currentYear
10
11          {return currentYear-startYear;}
12
13          public String toString(){
14                  return "Name:"+name+" job:"+job+" Starting-at:"+  startYear;}}
```

- When a method is defined as "final," this means it cannot be overridden.

The final Method error message

```
1   class SEmp  extends  Emp  implements  Company {
2           private double MSal;
3           public SEmp(String name, String job, int sy, double salary)
4                   {super(name,job,sy); this.MSal=salary;  }
5           public double Wsal() {
6               return MSal*12/52; }
7
8       public  int calYearServices(int currentYear)
9
10          {return currentYear-startYear;}
11
12          public String toString(){
13                  return super.toString()+"\n" + " His salary = "+MSal;
14  }
15  }
```

calYearServices(int) in SEmp cannot override calYearServices(int) in Emp; overridden method is final

The final Class

```
1   final class Emp {
2           protected String name;  protected String job;  protected int startYear;
3           public Emp(){ name="unknown"; job="unknown"; startYear=0;       }
4           public Emp(String name){ this(); this.name = name;}
5           public Emp(String name, String job) { this(name);this.job=job;}
6           public Emp(String name, String job, int sy){
7                this(name,job); startYear=sy;}
8
9           public  int calYearServices(int currentYear)
10
11          {return currentYear-startYear;}
12
13          public String toString(){
14                  return "Name:"+name+" job:"+job+" Starting-at:"+  startYear;}}
```

- When a class is defined as "final," this means it cannot be inherited.

The final Class error message

```
public class Trainee extends Emp {

    private int hours;

    public Trainee(String name, String job, int sy, int hours) {
        super(name, job, sy);
        this.hours = hours;
    }

    public String toString() {
        return super.toString() + "\nNo of Hours:" + hours;
    }
}
```

cannot inherit from final Emp

Inheritance Example

```java
package week.pkg10.lec.pkg1;

public class Employee {
    private String name;
    private int id;

    public Employee() {
    }

    public Employee(String name, int id) {
        this.name = name;
        this.id = id;
    }

    public String getName() {
        return name;
    }
```

```java
public int getId() {
    return id;
}

public void setName(String name) {
    this.name = name;
}

public void setId(int id) {
    this.id = id;
}

@Override
public String toString() {
    return "Employee{" + "name=" + name + ", id=" + id + '}';
}
```

```java
public double getTotalSal(){
return -1;
}
```

Inheritance Example

```java
package week.pkg10.lec.pkg1;

public class HEmp extends Employee{

private int noOfHours;
private double wage;

    public HEmp() {
        super();
    }

    public HEmp(int noOfHours, double wage, String name, int id) {
        super(name, id);
        this.noOfHours = noOfHours;
        this.wage = wage;
    }
```

```java
public int getNoOfHours() {
    return noOfHours;
}

public double getWage() {
    return wage;
}

public void setNoOfHours(int noOfHours) {
    this.noOfHours = noOfHours;
}

public void setWage(double wage) {
    this.wage = wage;
}
```

Inheritance Example

```java
@Override
    public double getTotalSal(){
    return this.noOfHours*this.wage;
    }
     public double getTotalSal(double T){
    return this.noOfHours*this.wage+T;
    }
     // Overloading
 public double getTotalSal(double T,double Tax){
    return this.noOfHours*this.wage+T-(this.noOfHours*this.wage*Tax);
    }
```

```java
public boolean equals(HEmp obj1)
{

if(this.noOfHours==obj1.noOfHours && this.wage==obj1.wage)
return true;
else
return false;




 }

}
```

Exercise (part 1)

- Using Java, Design and implement a class hierarchy to model different types of documents. Create a superclass named Document that encapsulates common attributes of all types of documents. Additionally, create a subclass named Book.
  - The superclass **Document** should include the following:
    - A **protected** instance variable **authorName** of type String.
    - Two **private** instance variables; **numberOfPages** of type int and **classificationNo** of type String.
    - A parameterized constructor to initialize the instance variables.
    - Appropriate getter and setter methods for the **numberOfPages** and **classificationNo** variables. The classificationNo attribute must start with the letter H or S (where H stands for hard copy documents and S for softcopy documents) followed by four digits such as "H1345", "S8756". Otherwise, you should display an error message (Invalid classification number).
    - An instance method named **display** that prints all relevant details of the **Document**.

Exercise (part 2)

- The subclass **Book** should inherit the instance variables and methods of the **Document** class. It should include the following:
  - **Private** instance variables for the book: **subject** of type String and **yearOfPublishing** of type int.
  - A parameterized constructor to initialize the instance variables.
  - Override the **display** method to display all details of the **book**.
  - An instance method called **checkYear** that checks if the book is published after a given year. The method should return **true** if it is published after or the same given year, and **false** otherwise.
  - Override the **equals** method to compare two books based on their subject. The method should return **true** if the subjects are the same, and **false** otherwise.

Exercise (part 3)

- Implement a test class (main method only) that performs the following:
  - Create two objects of type book with appropriate initial values of your choice.
  - Call the display method for both objects.
  - Check if the first object is equal to the second object. If the first object is equal to the second object, print **"Equal“,** otherwise, print **"Not Equal".**
  - Check and print the result of calling checkYear for both books objects with a year 2019.

References

- [1] Introduction to Java Programming, Comprehensive Version, Student Value Edition, Daniel Liang, 12th Edition, Pearson 2019
- lecture Notes: edugate.asu.edu.jo.
- lecture Videos: YouTube channel.
- Lab manual